

CSE422: Artificial Intelligence

Lab Project Report

Mushroom Classification Using Machine Learning

Dataset: Mushroom Dataset

Models Applied: KNN | Decision Tree | Neural Network | K-Means

Name	ID
Sahana Parvin Nupur	24141251
Ullas Sarker Tirtha	24141252

BRAC University

Department of Computer Science

Spring 2026

Table of Contents

1. Introduction	3
2. Dataset Description	4
2.1 Overview of the Dataset	4
2.2 Class Distribution	4
2.3 Feature Types and Encoding	5
2.4 Correlation Analysis	6
2.5 Exploratory Data Analysis (EDA)	7
3. Dataset Pre-Processing	10
3.1 Missing Values	10
3.2 Categorical Features	11
3.3 Feature Scaling	11
4. Dataset Splitting	12
5. Model Training & Testing	13
5.1 K-Nearest Neighbors (KNN)	13
5.2 Decision Tree	13
5.3 Neural Network (MLP)	13
5.4 K-Means Clustering	14
6. Model Selection & Comparison Analysis	15
7. Conclusion	19

1. Introduction

Mushrooms are a vital part of global ecosystems and cuisine, yet distinguishing edible mushrooms from poisonous ones remains a critical safety challenge. Misidentification can lead to severe poisoning or even death, making accurate classification of immense practical importance. This project aims to build and evaluate multiple machine learning models to classify mushrooms as either edible or poisonous based on a set of physical and environmental features.

The core problem being addressed is a binary classification task: given a set of observable mushroom characteristics (such as cap shape, gill color, habitat, and stem dimensions), can a machine learning model reliably predict whether the mushroom is safe to eat? This is a well-motivated real-world problem where a wrong prediction carries serious consequences.

The motivation behind this project is twofold. First, it provides a meaningful context to apply supervised learning techniques covered in CSE422, including K-Nearest Neighbors (KNN), Decision Tree, and Neural Network. Second, the dataset's mix of categorical and numerical features, combined with significant missing data, makes it an excellent testbed for practicing the complete machine learning pipeline - from data cleaning and encoding to model evaluation using metrics like precision, recall, AUC, and confusion matrices. Additionally, K-Means clustering is applied to treat the problem from an unsupervised learning perspective, reinforcing concepts taught in the K-Means lab.

2. Dataset Description

2.1 Overview of the Dataset

Property	Detail
Total Data Points (Rows)	61,069
Total Features (Columns)	21 (including target)
Problem Type	Binary Classification
Target Column	class (p = Poisonous, e = Edible)
Numerical Features	3 (cap-diameter, stem-height, stem-width)
Categorical Features	17 (cap-shape, cap-color, habitat, season, etc.)

The dataset contains 61,069 mushroom records with 20 input features and 1 target column (class). This is a classification problem because the goal is to assign each mushroom to one of two discrete categories: edible (e) or poisonous (p), rather than predicting a continuous value. The presence of a labeled output (the class column) confirms this is a supervised learning task.

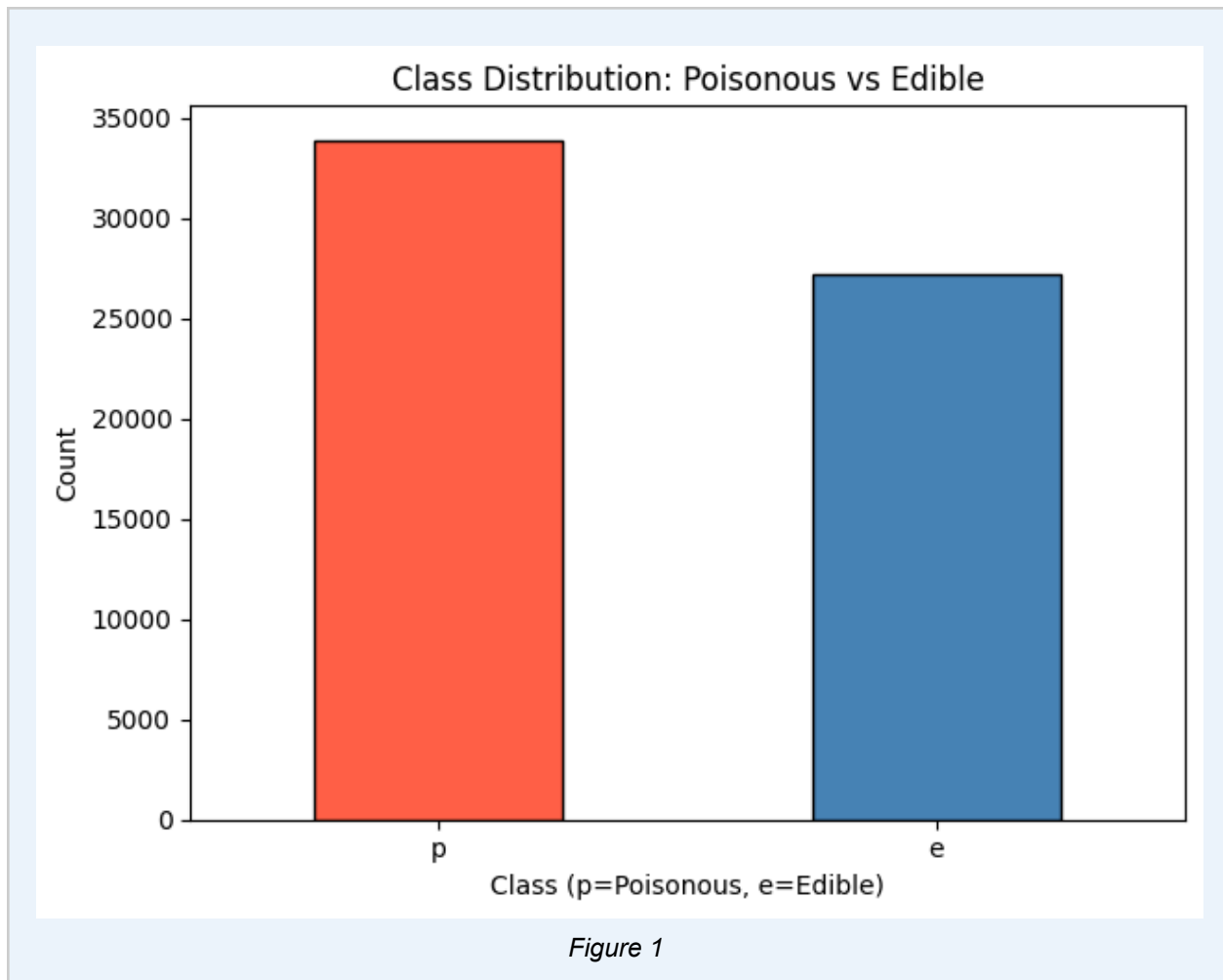
The features are a mix of quantitative (numerical) and categorical types. The three numerical features: cap-diameter, stem-height, and stem-width are continuous measurements. The remaining 17 features are categorical, describing physical attributes encoded as short string codes (e.g., cap-shape 'x' for convex, habitat 'd' for woods). Because machine learning algorithms require numerical inputs, all categorical variables must be encoded before model training.

2.2 Class Distribution (Imbalanced Dataset Check)

The output feature (class) has the following distribution:

Class	Label	Count	Percentage
p	Poisonous	33,888	55.5%
e	Edible	27,181	44.5%

The dataset is slightly imbalanced, with poisonous mushrooms making up approximately 55.5% of all records and edible mushrooms 44.5%. While this is not an extreme imbalance, it is worth noting. A model that always predicted 'poisonous' would achieve ~55.5% accuracy, so accuracy alone is not a sufficient metric. This motivates the use of stratified train/test splitting to preserve this ratio, as well as the evaluation of precision, recall, and AUC alongside accuracy.

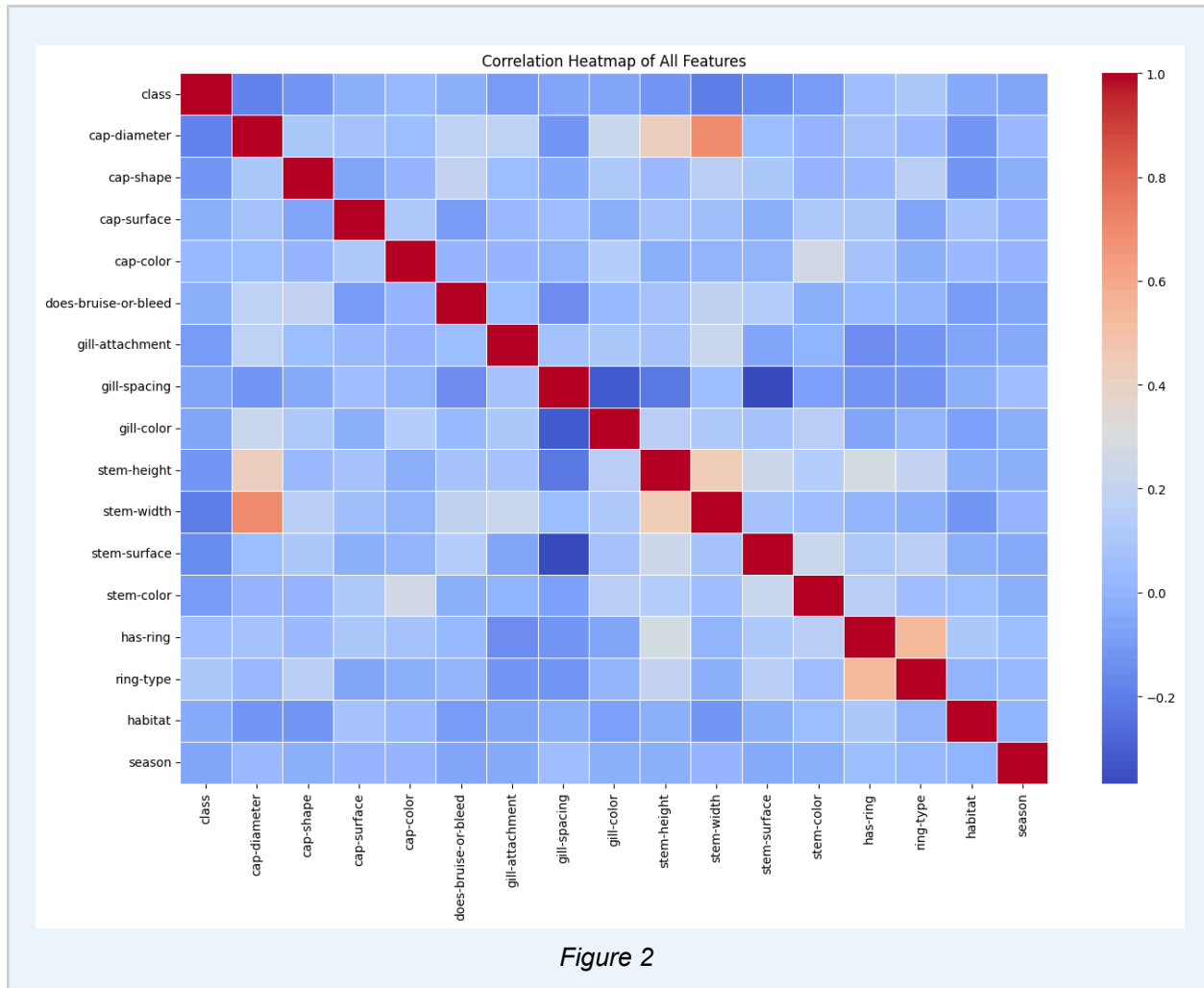


2.3 Feature Types and Encoding

Since 17 out of 20 input features are categorical (text-based), they cannot be used directly by machine learning algorithms. Label Encoding is applied to convert each unique category string into an integer. For example, in the 'cap-shape' column, values like 'x' (convex), 'f' (flat), 'b' (bell), etc., are each assigned a unique integer. This is appropriate here because we are not implying any ordinal relationship, the encoding is simply a necessary numerical representation.

2.4 Correlation Analysis

After encoding all features numerically, a Pearson correlation matrix was computed across all features including the target. The heatmap below visualizes pairwise correlations.



Key observations from the correlation analysis:

- stem-width ($r = -0.197$) and cap-diameter ($r = -0.181$) have the strongest correlations with the target class, meaning larger mushrooms tend to be edible in this dataset.
- stem-surface ($r = -0.154$) and stem-height ($r = -0.122$) also show moderate negative correlations with class.
- ring-type ($r = +0.092$) is the only feature with a positive correlation above 0.05, suggesting mushrooms with certain ring types are more likely to be poisonous.
- Most features have very weak correlations with the class label ($|r| < 0.1$), indicating that no single feature is highly predictive on its own, a combination of features is needed for accurate classification.
- has-ring and ring-type show a moderate positive correlation with each other ($r \sim 0.4$), which is expected.

2.5 Exploratory Data Analysis (EDA)

Several visualizations were produced to understand the structure and relationships in the data before modeling.

Numerical Feature Distributions

The histograms of the three numerical features reveal right-skewed distributions for all three. cap-diameter has most values concentrated between 0-15 cm with a long tail toward 60 cm. stem-height similarly peaks around 4-7 cm. stem-width follows a similar right-skewed pattern. The presence of extreme outliers suggests that Standard Scaler (which uses mean and std) may be influenced by these values, though it remains appropriate for KNN and Neural Networks.

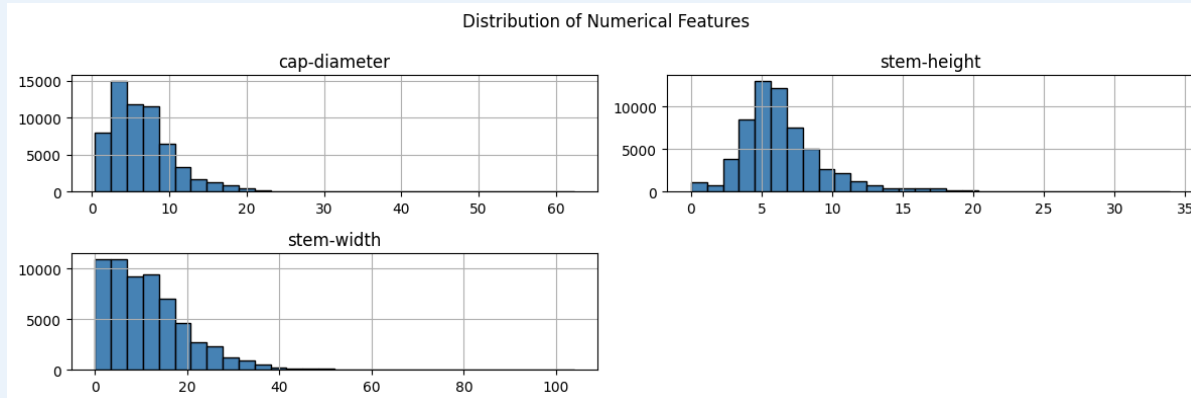
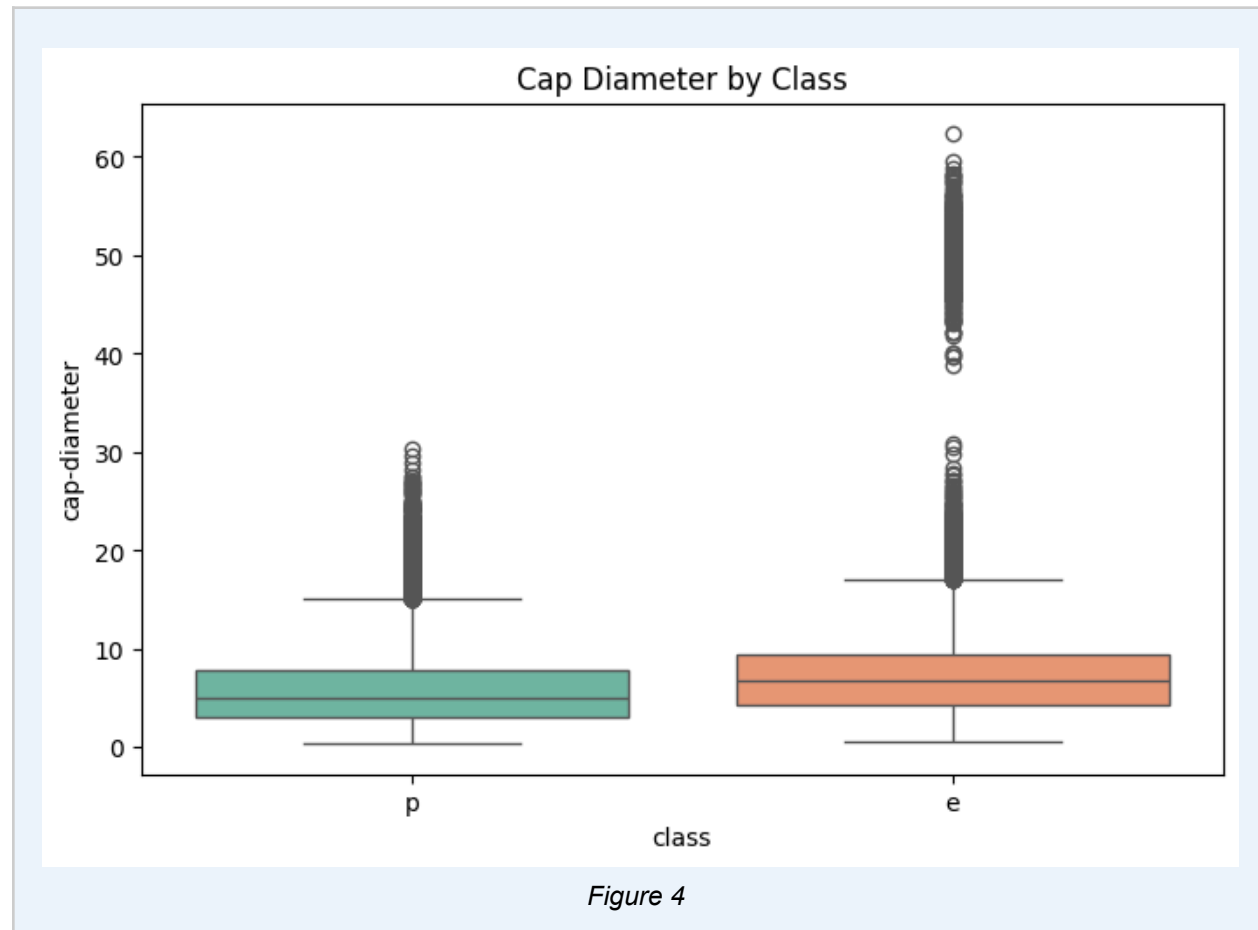


Figure 3

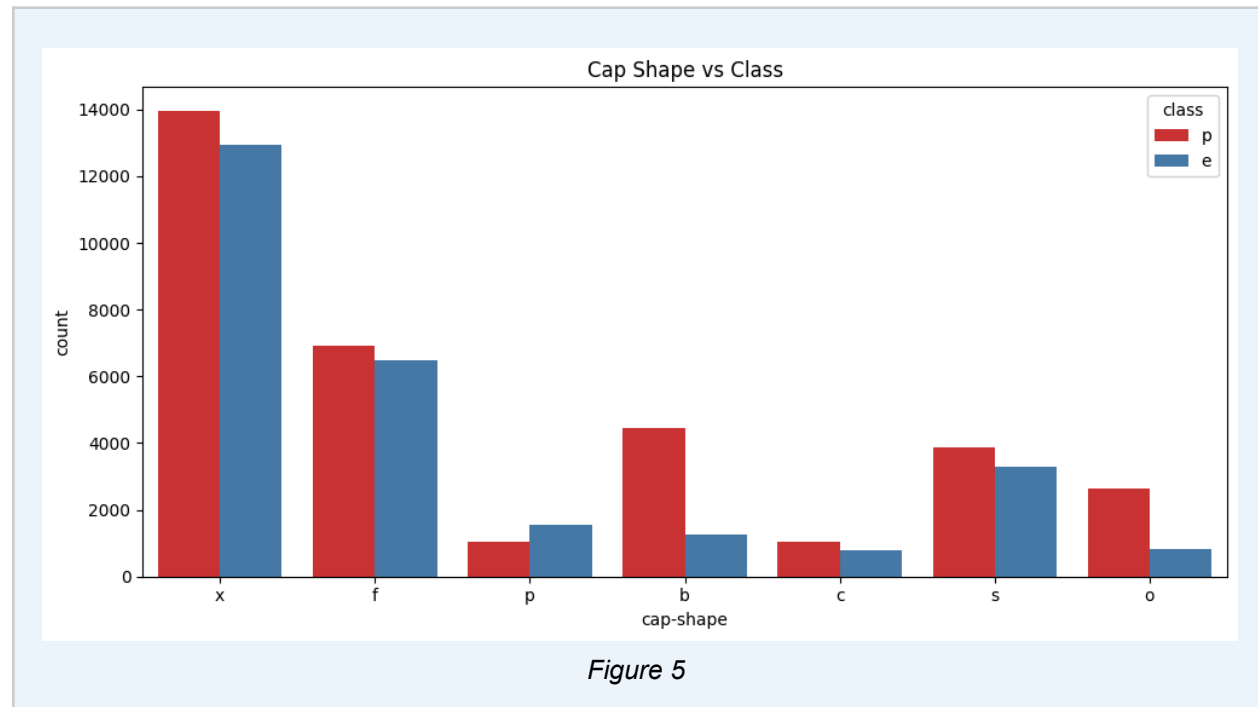
Cap Diameter by Class (Boxplot)

The boxplot comparing cap-diameter for poisonous vs. edible mushrooms shows that edible mushrooms (e) have a wider spread and more extreme outliers (some exceeding 60 cm), while poisonous mushrooms (p) are more tightly distributed with a median around 5 cm. This confirms the negative correlation seen in the heatmap, larger cap diameters tend to be associated with edible mushrooms.



Cap Shape vs Class (Countplot)

The countplot reveals that the convex cap shape ('x') is by far the most common for both classes. However, poisonous mushrooms dominate in the 'b' (bell) and 's' (sunken) shapes, while edible mushrooms have a slightly higher proportion in the 'p' (conical) shape. No cap shape is exclusively associated with one class, confirming that cap shape alone is insufficient for classification.



3. Dataset Pre-Processing

The raw dataset contained several issues that needed to be addressed before training any model. This section discusses each problem and the solution applied.

3.1 Missing Values

Problem

Nine features contained missing (NaN) values. The missing percentages ranged from ~4% to ~95%:

Feature	Missing Count	Missing %	Action Taken
veil-type	57,892	94.8%	Dropped
spore-print-color	54,715	89.6%	Dropped
veil-color	53,656	87.9%	Dropped
stem-root	51,538	84.4%	Dropped
stem-surface	38,124	62.4%	Imputed (mode)
gill-spacing	25,063	41.0%	Imputed (mode)
cap-surface	14,120	23.1%	Imputed (mode)
gill-attachment	9,884	16.2%	Imputed (mode)
ring-type	2,471	4.0%	Imputed (mode)

Solution

Columns with more than 80% missing values (veil-type, spore-print-color, veil-color, stem-root) were dropped entirely. Imputing more than half a column's values would introduce excessive artificial data and distort the model. The remaining five columns with missing values had their missing entries filled with the mode (most frequent value) of their respective column. Mode imputation is appropriate for categorical features as it preserves the most common category without creating new categories.

3.2 Categorical Features

Problem

After dropping the four highly-missing columns, 14 features (including the target 'class') remained as categorical string data. Machine learning algorithms require numerical inputs and cannot process raw text labels.

Solution

Label Encoding using sklearn's LabelEncoder was applied to all remaining categorical columns. Each unique string value in a column was mapped to a unique integer (e.g., for 'class': e=0, p=1). This was applied consistently to both training and test data. Label encoding is preferred over one-hot encoding here because the number of unique categories per feature is large enough that one-hot encoding would create an extremely high-dimensional and sparse feature matrix.

3.3 Feature Scaling

Problem

The numerical features (cap-diameter, stem-height, stem-width) have different scales and ranges. For distance-based algorithms like KNN and gradient-based algorithms like Neural Networks, features with larger magnitudes can dominate the computation, leading to biased results.

Solution

Standard Scaler (Z-score normalization) was applied to the dataset before training KNN and the Neural Network. This transforms each feature to have a mean of 0 and standard deviation of 1. The scaler was fitted only on the training data and then used to transform both the training and test sets, this prevents data leakage from the test set. Decision Tree does not require feature scaling since it makes decisions based on threshold splits, not distances.

4. Dataset Splitting

The dataset was split into a training set and a test set using stratified splitting:

Split	Size	Records
Training Set (80%)	80%	48,855
Test Set (20%)	20%	12,214

Stratified splitting was chosen over random splitting because the dataset is slightly imbalanced (55.5% poisonous, 44.5% edible). Stratified splitting ensures that the same class ratio is maintained in both the training and test sets. This prevents scenarios where the test set might accidentally contain a disproportionate number of one class, which would make evaluation metrics unreliable.

An 80/20 split was used. With 61,069 total records, 80% gives 48,855 training samples, more than sufficient for all three models to learn meaningful patterns. The 20% test set (12,214 samples) provides a large enough hold-out for statistically reliable evaluation metrics.

5. Model Training & Testing

5.1 K-Nearest Neighbors (KNN)

KNN is a simple, instance-based algorithm that classifies a new data point by looking at the K closest training examples (neighbors) in feature space and taking a majority vote. It is non-parametric, it makes no assumptions about the underlying data distribution and simply memorizes the training data.

Configuration: K=5 neighbors, Euclidean distance metric, with Standard Scaler applied beforehand. KNN is sensitive to feature scale, so scaling is essential. With K=5, each prediction is based on the 5 closest training points, providing a reasonable balance between noise sensitivity (low K) and over-smoothing (high K).

Result: KNN achieved a test accuracy of 99.94%. The confusion matrix shows only 7 misclassifications out of 12,214 test samples (3 edible predicted as poisonous, 4 poisonous predicted as edible).

5.2 Decision Tree

A Decision Tree recursively partitions the feature space by selecting the feature and threshold that best separates the classes at each node, measured by criteria like Gini impurity. It produces a tree of if-else rules that is highly interpretable, we can trace exactly why a sample was classified a certain way.

Configuration: max_depth=10 to prevent overfitting while allowing sufficient complexity. No feature scaling needed since Decision Trees are invariant to monotonic transformations of features.

Result: The Decision Tree achieved 92.50% accuracy. This is the lowest among the three models, with 916 misclassifications (458 per class). The limited depth (max_depth=10) restricts the tree from perfectly memorizing the training data, which explains the lower accuracy compared to KNN and NN. However, this also means the Decision Tree is likely more generalizable and explainable.

5.3 Neural Network (MLP)

The Multi-Layer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers with non-linear activation functions, and an output layer. It learns complex non-linear patterns through backpropagation and gradient descent.

Configuration: Two hidden layers of sizes (64, 32), ReLU activation, Standard Scaler applied, max 100 iterations. The network architecture (64-32-output) progressively compresses the representation, learning increasingly abstract features at each layer. Training stopped early at

iteration 65 when the loss improvement fell below the tolerance threshold ($\text{tol}=0.0001$ for 10 consecutive epochs), indicating convergence.

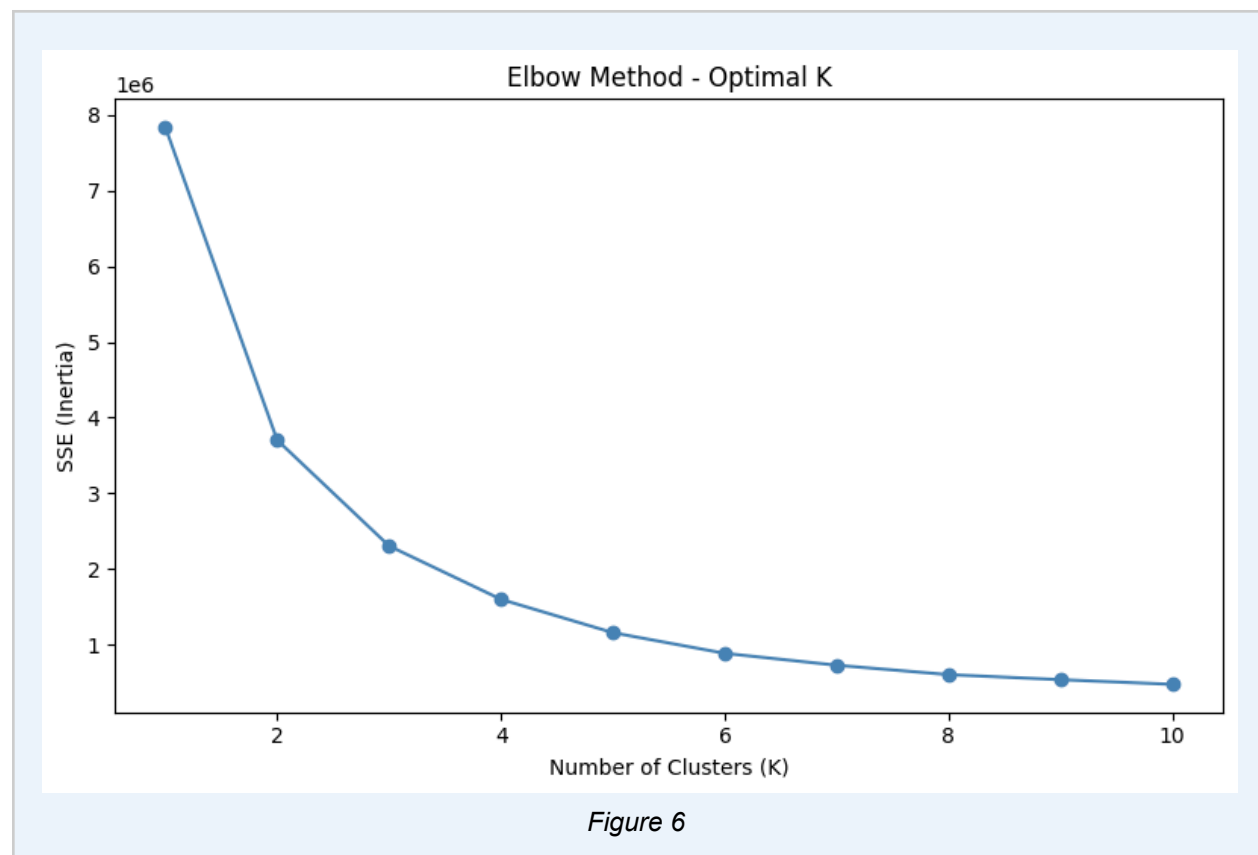
The training loss decreased from 0.4908 in iteration 1 to below 0.001 by iteration 40, demonstrating effective learning. Result: Neural Network achieved 99.975% accuracy, the highest of all three models with only 3 misclassifications (all poisonous mushrooms predicted as edible, 0 edible misclassified).

5.4 K-Means Clustering (Unsupervised)

To treat the problem from an unsupervised learning perspective, K-Means clustering was applied using cap-diameter and stem-width as the two features (chosen for easy 2D visualization). K-Means iteratively assigns data points to the nearest centroid and updates centroids until convergence.

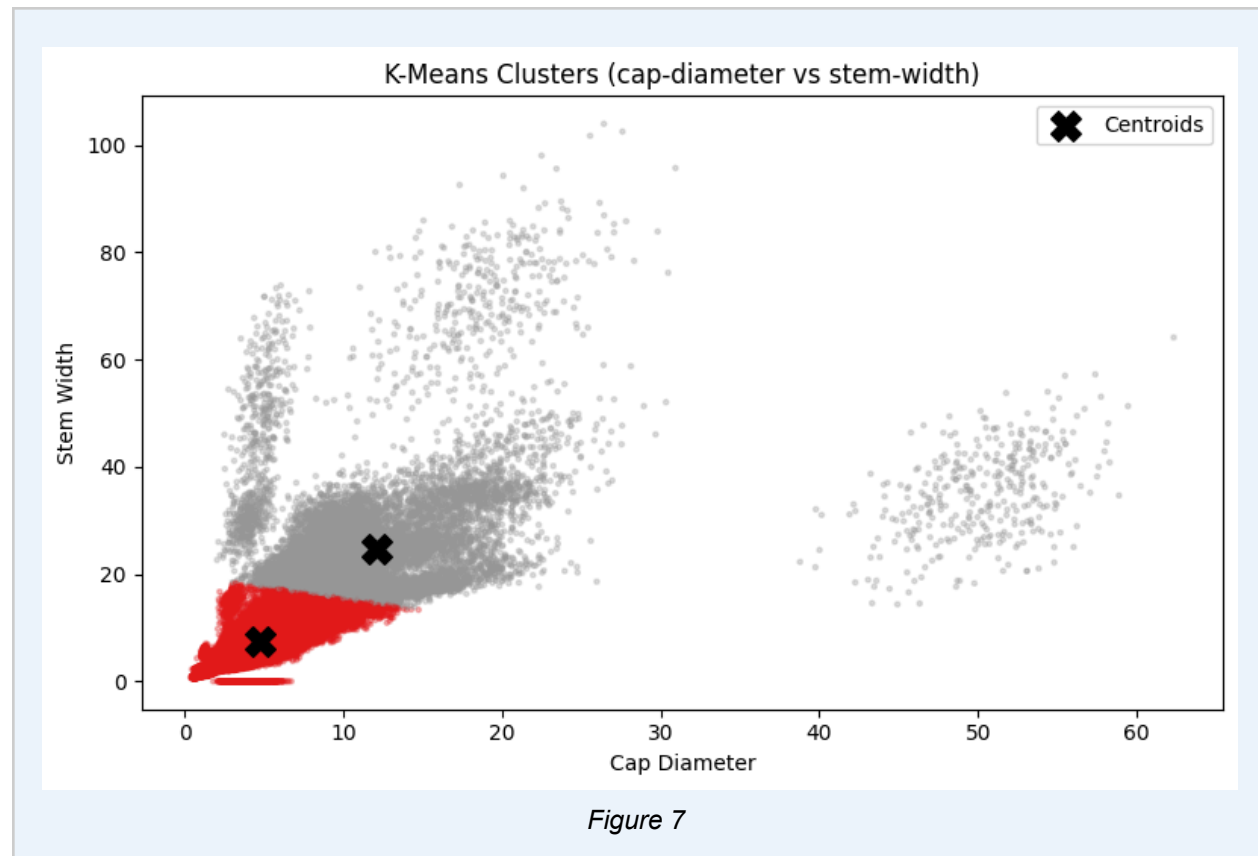
Elbow Method

The elbow method was used to determine the optimal number of clusters K . SSE (Sum of Squared Errors, also called inertia) was calculated for $K=1$ through $K=10$. The plot shows a steep drop from $K=1$ to $K=3$, after which the curve flattens. The 'elbow' is observed at approximately $K=2$ or $K=3$, which aligns with the ground truth of 2 natural classes (edible and poisonous). $K=2$ was selected for final clustering.



Cluster Visualization

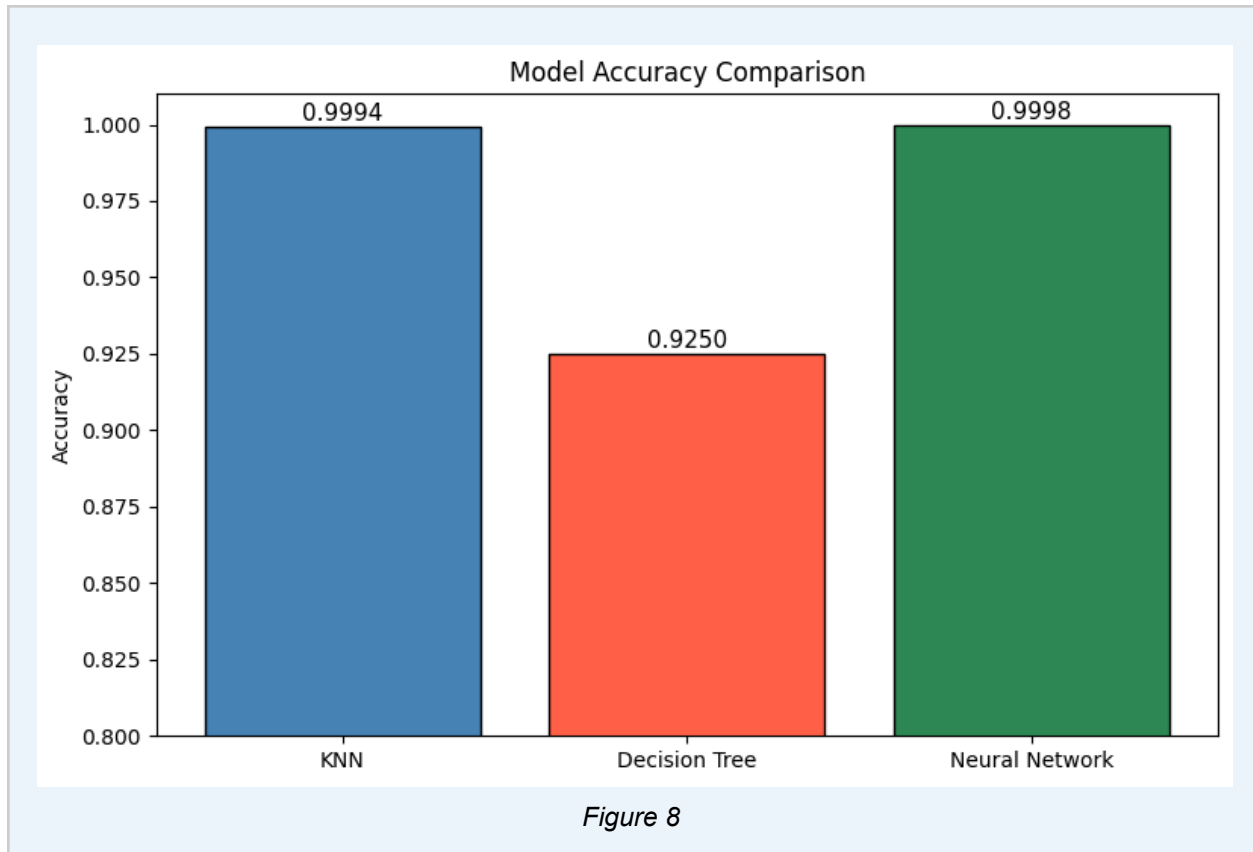
With $K=2$, the two clusters and their centroids are visualized in the scatter plot of cap-diameter vs. stem-width. The red cluster (Cluster 0) captures mushrooms with smaller cap diameters and stem widths, while the grey cluster (Cluster 1) captures larger specimens. The two centroids are marked with 'X' symbols. Although K-Means does not use the class labels, the clusters loosely correspond to the natural edible/poisonous groupings, validating that these physical dimensions do carry class-relevant information.



6. Model Selection & Comparison Analysis

6.1 Accuracy Comparison

Model	Accuracy	Correct Predictions	Misclassifications
KNN (K=5)	99.94%	12,207 / 12,214	7
Decision Tree (max_depth=10)	92.50%	11,298 / 12,214	916
Neural Network (MLP 64-32)	99.975%	12,211 / 12,214	3



6.2 Precision, Recall & F1-Score

Model	Class	Precision	Recall	F1-Score
KNN	Edible	1.00	1.00	1.00
KNN	Poisonous	1.00	1.00	1.00
Decision Tree	Edible	0.92	0.92	0.92
Decision Tree	Poisonous	0.93	0.93	0.93
Neural Network	Edible	1.00	1.00	1.00
Neural Network	Poisonous	1.00	1.00	1.00

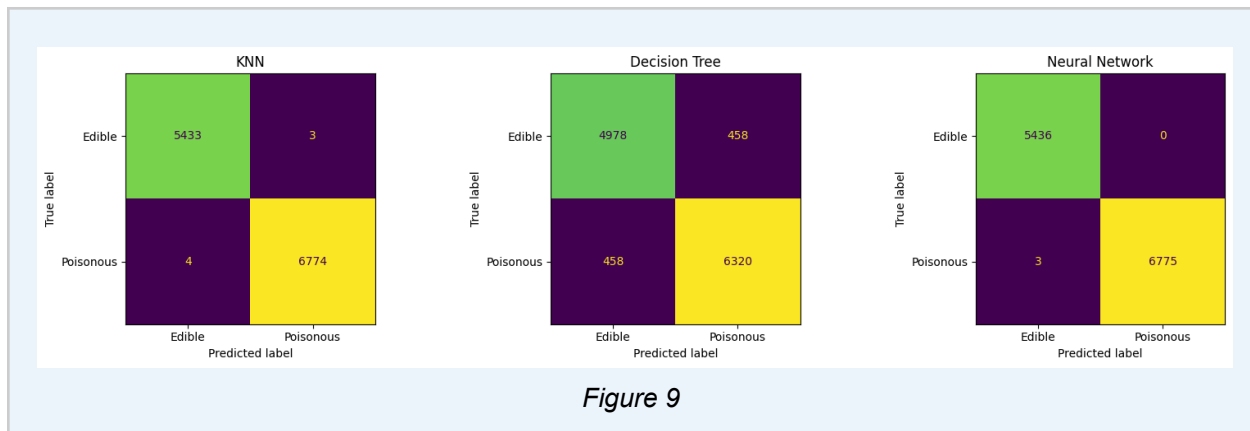
Both KNN and the Neural Network achieve near-perfect precision and recall across both classes. The Decision Tree, while still strong at ~92-93%, shows lower performance due to its depth constraint. High recall for the Poisonous class is especially important in this domain, missing a poisonous mushroom (False Negative) is more dangerous than incorrectly flagging an edible one.

6.3 Confusion Matrices

The confusion matrices reveal the exact nature of misclassifications for each model:

Model	True Edible Pred. Edible (TN)	True Edible Pred. Poisonous (FP)	True Poisonous Pred. Edible (FN)	True Poisonous Pred. Poisonous (TP)
KNN	5,433	3	4	6,774
Decision Tree	4,978	458	458	6,320
Neural Network	5,436	0	3	6,775

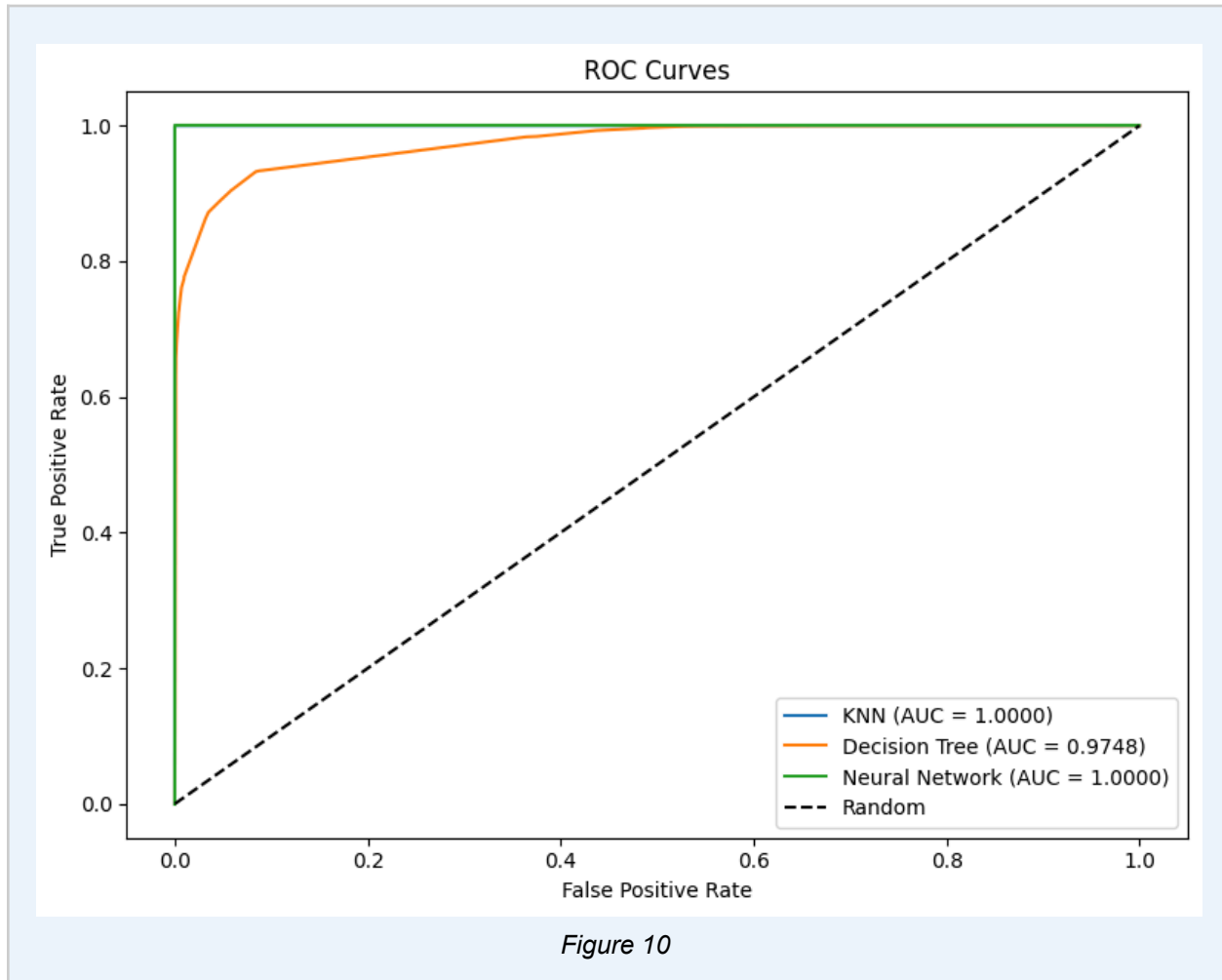
The Neural Network is particularly impressive in that it produced zero false positives (no edible mushroom was wrongly classified as poisonous) with only 3 false negatives. The Decision Tree's errors are evenly split between both types of misclassification (458 each), indicating balanced but imperfect performance. For a safety-critical application like mushroom identification, minimizing False Negatives (poisonous mushrooms classified as edible) is the priority.



6.4 ROC Curves & AUC Scores

Model	AUC Score
KNN	1.0000
Decision Tree	0.9748
Neural Network	1.0000

The ROC curves plot the True Positive Rate against the False Positive Rate at various classification thresholds. An AUC of 1.0 represents a perfect classifier. Both KNN and the Neural Network achieve a perfect AUC of 1.0, meaning they can perfectly rank positive (poisonous) samples above negative (edible) ones regardless of threshold. The Decision Tree achieves a strong AUC of 0.9748, indicating it is still a very reliable discriminator despite the lower accuracy. The Decision Tree's ROC curve visibly diverges from the top-left corner, particularly in the low false-positive-rate region.



7. Conclusion

7.1 Summary of Results

This project successfully applied three supervised machine learning models and one unsupervised clustering algorithm to classify mushrooms as edible or poisonous from the UCI Mushroom dataset (61,069 samples, 20 features). The results demonstrate that machine learning is highly effective for this classification problem, with two of the three models achieving near-perfect accuracy.

The Neural Network (MLP with hidden layers 64-32) achieved the best overall performance with 99.975% accuracy and a perfect AUC of 1.0. Remarkably, it produced zero false positives, no edible mushroom was incorrectly classified as poisonous. KNN (K=5) was a very close second with 99.94% accuracy and AUC of 1.0, with only 7 total misclassifications. The Decision Tree, constrained to `max_depth=10`, achieved 92.50% accuracy and AUC of 0.9748, still excellent, but noticeably lower than the other two.

7.2 Interpretation of Results

The near-perfect performance of KNN and the Neural Network suggests that the mushroom features, after proper preprocessing (imputation, encoding, and scaling), are highly discriminative. The structured combination of physical features (stem dimensions, gill properties, cap attributes) creates clear separable patterns in the feature space that both distance-based (KNN) and representation-learning (Neural Network) approaches can exploit.

The Decision Tree's lower performance is attributable to the `max_depth=10` constraint, which limits its ability to fully capture the complex decision boundaries in the data. Without this constraint, a fully grown Decision Tree would likely achieve higher accuracy but would risk overfitting. This tradeoff between bias and variance is well illustrated by comparing the Decision Tree to KNN and the Neural Network.

From the K-Means analysis, the elbow method pointed to K=2 as the natural number of clusters, which aligns perfectly with the true number of classes. This validates that the physical features of mushrooms do create natural groupings that correspond to toxicity, an encouraging finding that supports the biological basis of the classification.

7.3 Challenges Faced

- **Missing Data:** A significant challenge was the extensive missing data, four columns had over 80% of their values missing. Simply imputing them would have introduced substantial noise, so they were dropped. Deciding which columns to drop vs. impute required careful analysis of missingness rates and feature importance.
- **Categorical Encoding:** With 17 categorical features, encoding was non-trivial. Label encoding was chosen over one-hot encoding to avoid extreme feature space expansion, but this means the model may implicitly assume ordinal relationships between encoded values where none exist.
- **Model Selection Tradeoff:** While KNN and Neural Network performed nearly perfectly, this raises a question about whether the dataset is 'too easy' for these models, potentially indicating that the data may be somewhat synthetic or that the features are highly discriminative. In a real-world deployment, testing on truly wild mushroom samples would be critical.
- **Computational Cost:** KNN with $K=5$ on a dataset of ~49,000 training samples requires computing distances to all training points at prediction time, making it computationally expensive. The Neural Network, once trained, is much faster at inference.
- **Class Imbalance:** Although mild (55.5% vs. 44.5%), the class imbalance required the use of stratified splitting and careful attention to per-class metrics (precision, recall) rather than relying solely on overall accuracy.

7.4 Final Remarks

For a safety-critical application such as real-world mushroom identification, we would recommend the Neural Network as the primary model due to its perfect AUC and zero false positive rate in this experiment. However, it is important to acknowledge that all models were evaluated on a held-out subset of the same dataset. Deployment on truly novel mushroom samples would require extensive additional validation. Future work could explore ensemble methods, feature importance analysis, and collection of more diverse real-world data to build a more robust classifier.

— End of Report —